

SECCIÓN 1.1

DESARROLLO WEB ENTORNO **SERVIDOR** 2ºDAW



CIPFP AUSIÀS MARCH
CENTRE INTEGRAT PÚBLIC
DE FORMACIÓ PROFESSIONAL



INTRODUCCIÓN A NODE.JS

Características principales:

- Permite ejecutar código JavaScript **fuera del navegador**.
- Diseñado para ejecutarse en el servidor o en cualquier máquina local.
- Funciona como un lenguaje de programación backend tradicional (similar me a Python, Java o PHP).



JAVASCRIPT: NAVEGADOR VS. SERVIDOR



JavaScript en el Navegador (Frontend):

- Manipulación del DOM y de la interfaz de usuario.
- Gestión de eventos, modales y efectos visuales.
- **Limitaciones:** Acceso restringido por seguridad (no puede acceder al sistema de archivos local).

JavaScript con Node.js (Backend):

- Sin interfaz visual ni DOM (no existe el objeto document o window).
- Acceso directo al sistema operativo y recursos de la máquina.

**El DOM (Document Object Model) es la representación que hace el navegador de una página HTML en forma de árbol de objetos, para que JavaScript pueda acceder y modificarla.*

ARQUITECTURA INTERNA: EL MOTOR V8



■ ¿Qué es V8?

- Es el motor de JavaScript desarrollado por **Google (usado en Chrome)**.
- Escrito en C++.



■ Función del motor V8:

- Toma el código JavaScript y lo **compila a código máquina** directamente para ejecutarse de forma rápida y eficiente.

■ Node.js + V8:

- Node.js utiliza V8 en su núcleo, pero le añade módulos adicionales para ampliar sus capacidades.

¿QUÉ AÑADE NODE.JS A JAVASCRIPT?



■ Nuevas funcionalidades integradas:

- **Acceso al sistema de archivos (File System):** Lectura, escritura, creación y borrado de archivos locales.
- **Redes y Servidores:** Creación de servidores HTTP, manejo de sockets.
- **Manejo de Procesos:** Gestión del sistema operativo local.

Toma la velocidad de V8 y le añade la potencia de acceder al sistema donde se ejecuta.

■ Puntos clave:

- Node.js **no** es un nuevo lenguaje, es un entorno de ejecución para JavaScript.
- Permite usar un único lenguaje (JavaScript) tanto en el frontend como en el backend.
- Pierde la manipulación del DOM, pero gana capacidades de servidor y acceso a archivos.

INSTALACIÓN Y VERIFICACIÓN DE NODE.JS



- **Descarga:** Desde la web oficial (nodejs.org).
 - Se recomienda instalar la versión LTS (soporte a largo plazo) o la más reciente.
- **Proceso:** Instalador estándar para Windows, macOS y Linux (mantener opciones por defecto).
- **Verificación en Terminal:**
 - Comando: `node -v`
 - Muestra la versión instalada para confirmar que el entorno está listo.

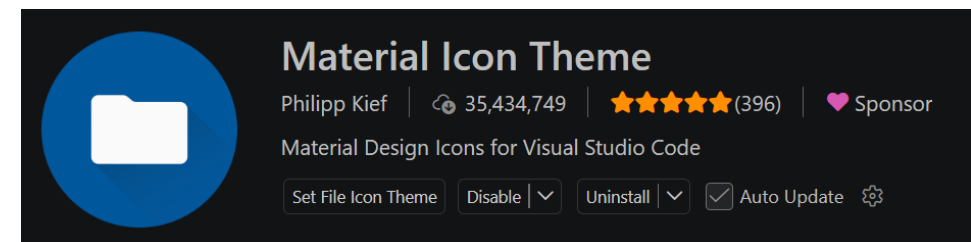
```
C:\Users\carlo>node -v
v24.20.0
```

```
C:\Users\carlo>node
Welcome to Node.js v24.20.0.
Type ".help" for more information.
> 1+1
2
```

ENTORNO DE DESARROLLO (IDE)



- **Visual Studio Code (VS Code):** Editor recomendado y estándar en la industria.
- **Configuración del entorno:**
 - **Integración:** Incluye terminal integrada (Terminal > New Terminal).
 - **Estructura:** Trabajar con carpetas de proyecto organizadas.
 - **Extensiones opcionales:** Temas visuales e iconos (*Material Icon Theme*) para mejorar la legibilidad del código.



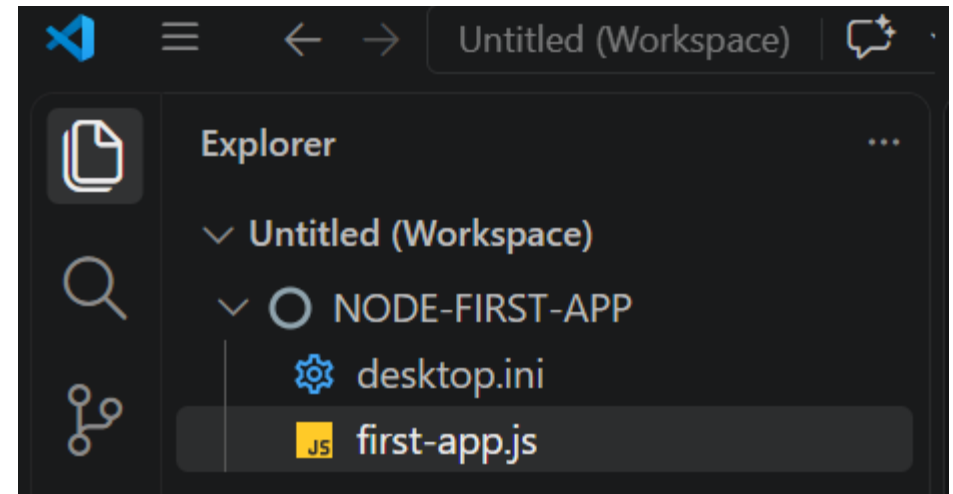
MODOS DE EJECUCIÓN DE NODE.JS

■ Modo Interactivo (REPL):

- Se activa escribiendo node en la terminal.
- Permite ejecutar JS de forma rápida o usar la consola como calculadora.
- *Uso:* Pruebas rápidas (no se usa para aplicaciones reales).
- Ctrl C dos veces para salir

■ Modo Script (Ejecución de Archivos):

- Escribir el código en un archivo .js (ejemplo: app.js).
- Ejecutar desde la terminal mediante el comando: node nombre_archivo.js



SECCIÓN I - EJEMPLO I:

JavaScript Básico vs Acceso al Sistema

Parte I:

- `console.log('¡Hola desde Node.js!');`

```
Cliente 2DAW\Node\NODE-FIRST-APP> node first-app.js
```

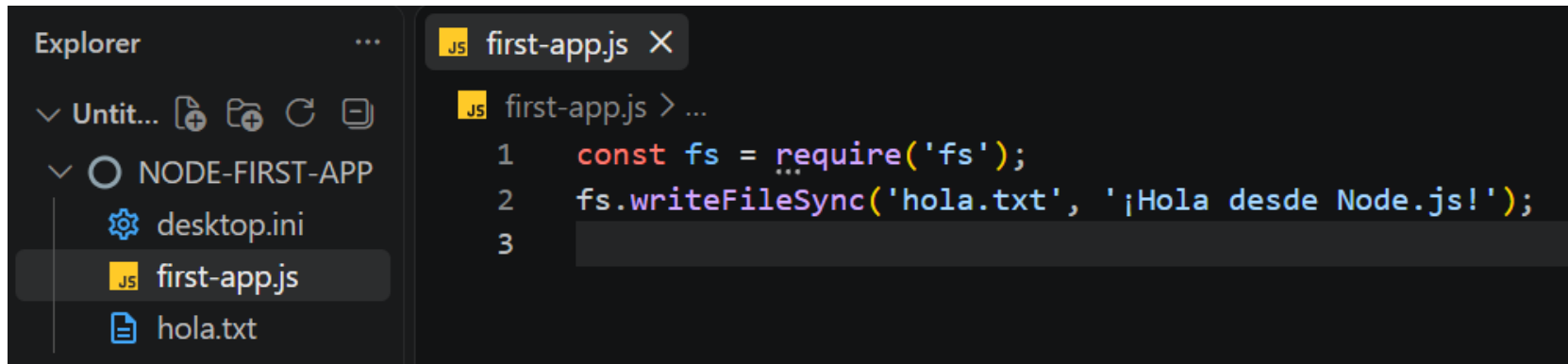
```
Hola desde Node.js
```

SECCIÓN I - EJEMPLO I:

Parte 2:

- `const fs = require('fs');`
- `fs.writeFileSync('hola.txt', '¡Hola desde Node.js!');`

En el navegador no podemos crear archivos locales en el disco duro por razones de seguridad; en Node.js usamos el módulo **fs (File System)** para hacerlo directamente.



```
JS first-app.js X
JS first-app.js > ...
1  const fs = require('fs');
2  fs.writeFileSync('hola.txt', '¡Hola desde Node.js!');
3
```

RESUMEN: CONFIGURACIÓN INICIAL



Node.js se ejecuta desde la terminal mediante scripts .js.

Combina sintaxis JS estándar (console.log) con capacidades de servidor (require('fs')).

El entorno está preparado para comenzar con la creación de aplicaciones
Backend completas.